

PROJECT 14

Personal Podcast Platform

Software Engineering Documentation

Team Members	Andi Morina, Mert Boza, Rinor Nuredini, Taulant Rama, Valëza Sutaj
Website	GjirafaPodcast
Login credentials for testing AdminDashboard	Email : taulant@gmail.com Password : taulant74
Login credentials for testing PublisherDashboard	Email : publisher@gmail.com Password : taulant74
Repository	github.com/Taulant74/Personal Podcast

1. Software Engineering Requirements

1.1 Feature Specifications

All features implemented in the Personal Podcast Platform are described below with full detail on expected behavior, acceptance criteria, business value, and priority.

Feature	Priority	Acceptance Criteria	Business Value
Home Page - Episode Listing	High	Displays all published episodes with title, description, duration, and categories. Episodes load with pagination.	Core discovery mechanism for listeners. First impression of the platform.
Episode Player	High	Audio plays free episodes , while premium episodes need to be ordered. Play count increments on each play. Supports pause/resume.	Fundamental value of a podcast platform. Enables content consumption.
Episode Ordering	High	Authenticated users can order an premium episode once. Duplicate orders rejected.	Enables users to build a personal library of ordered episodes.
Search & Pagination	High	Search by title, description, or category. Paginated results. Returns matching episodes.	Essential for platforms with large content libraries.
User Authentication (JWT)	High	Register and login with JWT access token + refresh cookie. Password min 8 chars, 1 letter, 1 number. Roles: User, Publisher, Admin.	Required for personalized features and content protection.
Episode Categories / Seasons	High	Episodes tagged with one or more categories. Filter by category on CategoryPage. Season number tracked.	Improves content organization and discoverability.
Episode Details & Metadata	High	Episode page shows title, description, duration, publish date, play speed , play count, categories.	Informs users before committing to listening.
Publish / Unpublish Episodes	High	Publisher can toggle IsPublished. Unpublished episodes hidden from public. Admin can override.	Content lifecycle management for creators.

Feature	Priority	Acceptance Criteria	Business Value
Publisher Dashboard	Medium	Publishers see and manage their own episodes. Can upload new episodes with Cloudinary audio URL.	Creator self-service reduces admin workload.
Admin Dashboard	Medium	Admin manages all users (CRUD) and all episodes. Role assignment. Full CRUD on categories.	Platform governance and content quality control.
User Profile Panel	Medium	Users view/edit their profile (username, name, email, age). View ordered episodes.	Personalization and account management.
My Episodes / Play History	Medium	Authenticated users see all episodes they have ordered. Acts as play history.	Supports re-listening and user retention.
Episode Upload (Cloudinary)	Medium	Publisher uploads audio to Cloudinary. URL stored in DB. Supports image thumbnail.	Removes server storage burden. Scalable media delivery.
Protected Routes & Roles	High	Routes enforce role via RequireRole component. Unauthenticated users redirected to login.	Security and proper access control.

1.2 User Stories

Feature: Episode Browsing & Playing

ID	User Story	Acceptance Criteria	Tasks
US-01	As a listener, I want to see a list of podcast episodes on the home page so I can discover content.	Home page loads published episodes. Each card shows title, duration, and category tags. Pagination controls visible.	Implement GET /api/getepisodes endpoint with pagination. Build HomePage.jsx episode grid.
US-02	As a listener, I want to play an episode audio directly on the platform.	Click open on episode card for the episode form to show up where you can play the episode. CompactAudioPlayer renders. Play count increments via POST /api/episodes/{id}/play.	Build CompactAudioPlayer component. Integrate with EpisodePage. Implement play count increment API.

ID	User Story	Acceptance Criteria	Tasks
US-03	As a listener, I want to search for episodes by keyword so I can find specific content.	Search bar on the episodes page. Results filter by title, description, or category name. Empty state shown for no results.	Implement GET /api/episodes/search . Build search UI .
US-04	As a listener, I want to view episode details including description and metadata.	Click open on episode card for episode form to show up where you can view episode details.	Fetch episode by ID from GET /api/getepisodes/{id}.

Feature: User Authentication

ID	User Story	Acceptance Criteria	Tasks
US-05	As a new user, I want to register an account so I can access personalized features.	Registration form with username, first name, last name, password. Password validation enforced. JWT returned on success.	Implement POST /api/auth/register. Build RegisterPage.jsx. Add AuthContext with token storage.
US-06	As a registered user, I want to log in to access my saved episodes and profile.	Login form accepts username or email + password. On success, access token set in context, refresh cookie set. Redirect to home.	Implement POST /api/auth/login. Build LoginPage.jsx. Implement token refresh logic.
US-07	As an authenticated user, I want my session to persist so I don't have to log in every time.	Refresh token cookie valid 7 days. POST /api/auth/refresh returns new access token. AuthContext handles auto-refresh.	Implement POST /api/auth/refresh. Update AuthContext to call refresh on 401 responses.

Feature: Episode Ordering & Library

ID	User Story	Acceptance Criteria	Tasks
US-08	As a logged-in listener, I want to order a premium episode to my personal library.	Order button on episode card. POST /api/orders creates order. Duplicate prevented (409). Order appears in My Episodes.	Implement POST /api/orders. Build order button in UI. Add duplicate check.
US-09	As a listener, I want to view all episodes I have ordered.	My Episodes page lists all ordered episodes	Implement GET /api/orders/my-episodes. Build MyEpisodesPage.jsx.

Feature: Publisher Features

ID	User Story	Acceptance Criteria	Tasks
US-10	As a publisher, I want to upload a new episode so listeners can discover it.	Upload form accepts title, description, audio URL (Cloudinary) ,season, categories.,Is Premium = false by default, saved with IsPublished=false by default.	Integrate Cloudinary upload. Build EpisodesUpload.jsx. Implement POST in PublisherController.
US-11	As a publisher, I want to manage my own episodes (edit, delete, publish/unpublish).	Publisher dashboard lists only their episodes. Edit and delete buttons present. Toggle publish state.	Build PublisherDashboard.jsx. Implement PUT/DELETE in PublisherController. Restrict to own episodes.

Feature: Admin Features

ID	User Story	Acceptance Criteria	Tasks
US-12	As an admin, I want to manage all users on the platform (view, create, edit, delete, change role).	Admin dashboard shows all users. CRUD operations functional. Role can be set to User, Publisher, or Admin.	Build AdminDashboard.jsx user management tab. Implement CRUD in AdminController.
US-13	As an admin, I want to manage all episodes regardless of publisher.	Admin can edit, delete, publish/unpublish any episode. All episodes visible regardless of IsPublished state.	Build episode management tab in AdminDashboard. Full episode CRUD in AdminController.
US-14	As an admin, I want to manage episode categories.	Admin can create new categories .	Implement POST/api/categories. Build category management UI in AdminDashboard.

Feature: Episode Categories / Seasons

ID	User Story	Acceptance Criteria	Tasks
US-15	As a listener, I want to filter episodes by category so I can quickly browse topics I like.	Category page (or category filter) shows only episodes that match the selected category. Pagination still works. If no results, show an empty state message.	Implement category filtering in GET /api/getepisodes (e.g., ?category=...) or add GET /api/categories/{id}/episodes. Build CategoryPage.jsx + category dropdown on episodes page.

Feature: Protected Routes & Roles

ID	User Story	Acceptance Criteria	Tasks
US-16	As a user, I want the app to block pages I'm not allowed to	If user is not logged in → redirected to Login. If user has wrong role → show "Unauthorized" (or redirect).	Add RequireRole wrapper in React Router routes. Enforce [Authorize(Roles="...")] on backend controllers. Add

	access, so my account stays secure.	Admin pages work only for Admin; Publisher pages only for Publisher/Admin.	frontend "Unauthorized" page/component.
--	-------------------------------------	--	---

Feature: User Profile Panel

ID	User Story	Acceptance Criteria	Tasks
US-17	As a user, I want to view my profile information so I can see my account details.	Profile page displays username, name, email, age, and role. Only logged-in users can access it.	Implement GET /api/user/profile (or similar). Build ProfilePage.jsx and protect route

Feature: Episode Player (Premium Restriction)

ID	User Story	Acceptance Criteria	Tasks
US-18	As a publisher, I want my free episodes to be played for free but premium episodes to be ordered so i can earn money from selling them.	Free episode plays instantly. Premium episode: if not ordered → show "Order to play" and block playback. If ordered → playback works normally.	Add isPremium check in frontend player logic. Add backend check endpoint (e.g., GET /api/orders/has/{episodeId}) or return "canPlay" in episode details response. Update UI to show "Order" button when blocked.

1.3 Agile / Scrum Implementation

Sprint 1 (13 Feb - 21 Feb : Authentication, Basic MVP)

Title	Points	Description
Authentication	6 pts	JWT-based register/login/refresh. BCrypt password hashing. Role-based claims (User, Publisher, Admin).
Search & Pagination	4 pts	Episode search by title/description/category. Paginated GET /api/episodes/search endpoint.
Episode Details & Player	5 pts	EpisodePage.jsx with full metadata. CompactAudioPlayer component. Play count tracking.
DevOps, Filtering & Cloudinary	5 pts	Cloudinary integration for audio uploads. Initial CI/CD pipeline setup. Category filtering.
Admin Dashboard	3 pts	Admin user management (CRUD). Role assignment. Category management.

Sprint 1 Goal: Deliver a working local MVP including authentication, episode browsing, search functionality, and containerized setup.

Sprint 1 Review: All 5 items delivered. Authentication is fully functional with JWT + refresh token. Cloudinary integrated. Admin dashboard operational. Search and pagination working end-to-end. Episode detail page with audio player complete.

Sprint 2 (21 Feb - 28 Feb : Production-Ready Release)

Title	Points	Description
UI Polish & Navigation	4 pts	UI polishing across all pages. Clean code refactoring. User navigation improvements. User panels.
Profile, Categories, Orders	5 pts	Profile page (UserPanelPage). CategoryPage with filtered episodes. Order system (save/library).
DevOps & Hosting	6 pts	Azure Kubernetes deployment. Jenkins CI/CD pipeline. Dockerfiles for FE and BE. k8s manifests.
Protected Routes & Roles	3 pts	RequireRole component. Route protection for Publisher and Admin pages. Token refresh logic.
Testing + Documentation + Extras	6 pts	Unit tests for search, auth, order logic. SE documentation. Code quality review.

Sprint 2 Goal: Production-ready release : secure, tested, documented, deployed on Azure Kubernetes with CI/CD.

Sprint 2 Review: Ongoing (sprint dates 23–26 Feb). POD-49 and POD-50 currently In Progress. POD-51, POD-52, POD-53 queued for completion.

2. Product Backlog & Prioritization

Feature	Priority	Acceptance Criteria	Business Value Justification	Epics
Core Public Pages & Navigation	High	Home Page and Episodes Page displays episodes ,About page shows project info,Help page provides usage guidance,Navigation works correctly	Provides public-facing entry points and information architecture for users.	Epic 6 - UI Quality & User Experience
Audio Player	High	Episode audio plays inline, play count tracked	The primary use case of a podcast platform - without	Epic 2 - Episode Management & Content Lifecycle

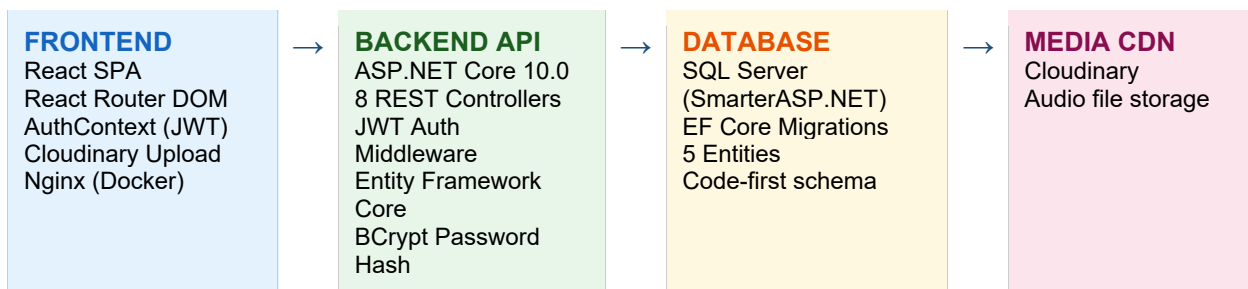
Feature	Priority	Acceptance Criteria	Business Value Justification	Epics
			playback it has no value.	
JWT Authentication	High	Register, login, refresh token. BCrypt hashing. Role claims.	Enables personalized features, content management, and security.	Epic 1 - User Authentication & Access Control
Search & Pagination	High	Search by keyword across title, description, category	Essential at any scale. Users cannot find content without search.	Epic 3 - Discovery & Search
Episode Categories	High	Episodes tagged. Filter by category. Many-to-many relation.	Organizes content. Improves discoverability and reduces bounce rate.	Epic 3 - Discovery & Search
Episode Ordering / Library	High	Authenticated users order premium episodes. Duplicate prevention.	Drives user retention. Personal libraries increase session length.	Epic 4 - User Personalization
Publish / Unpublish	High	Publisher controls episode visibility via IsPublished flag	Essential for content lifecycle. Protects against incomplete uploads.	Epic 2 - Episode Management & Content Lifecycle
Role-Based Access Control	High	Roles: User, Publisher, Admin. Protected routes. JWT claims.	Security requirement. Without RBAC, any user could modify content.	Epic 1 - User Authentication & Access Control
Admin Dashboard	Medium	Full user CRUD, episode management, category management	Operational necessity for platform governance and moderation.	Epic 2 - Episode Management & Content Lifecycle
Publisher Dashboard	Medium	Publishers manage only their own episodes. Upload + Cloudinary.	Self-service content management reduces admin dependency.	Epic 2 - Episode Management & Content Lifecycle
User Profile Panel	Medium	Edit username, name, email, age. View ordered episodes.	Personalization increases user satisfaction and platform stickiness.	Epic 4 - User Personalization

Feature	Priority	Acceptance Criteria	Business Value Justification	Epics
Cloudinary Integration	Medium	Audio files hosted on Cloudinary. URL stored in DB.	Removes server storage burden. Enables scalable media delivery.	Epic 5 - Platform Infrastructure & DevOps
Azure Kubernetes Deployment	High	FE and BE deployed as separate AKS pods. Jenkins CI/CD.	Meets infrastructure requirement. Production reliability.	Epic 5 - Platform Infrastructure & DevOps
Unit Tests//53	Medium	Min 20% coverage on search, auth, order logic	Code quality and regression prevention.	Epic 5 - Platform Infrastructure & DevOps

3. Architecture & Design

3.1 System Architecture

The Personal Podcast Platform follows a three-tier architecture with a clear separation between the presentation layer (React SPA), the application layer (ASP.NET Core Web API), and the data layer (SQL Server via Entity Framework Core). Media storage is offloaded to Cloudinary CDN.



Deployment layer: Both frontend and backend are containerized with Docker and deployed as separate pods on Azure Kubernetes Service (AKS). Jenkins handles CI/CD pipeline builds. Kubernetes manifests are in the /k8s directory.

3.2 Data Model

Entity	Key Fields	Type	Constraints	Relationships
User	Id, Username, FirstName, LastName, Age,	Identity & Auth	Username is unique. Password BCrypt hashed. Role ∈	HasMany: Orders, Episodes (as Publisher)

Entity	Key Fields	Type	Constraints	Relationships
Episode	Email, Role, PasswordHash, PasswordSalt, CreatedAt	Core Content	{User, Publisher, Admin}	BelongsTo: User (Publisher). HasMany: EpisodeCategory, Order
	Id, Title, Description, AudioUrl, DurationSeconds, Season, IsPublished, PublishedDate, IsPremium, PlayCount, PublisherId, CreatedAt		AudioUrl required. IsPublished defaults false. PlayCount ≥ 0	
Category	Id, Name	Taxonomy	Name unique (case-insensitive). Cannot delete if in use.	HasMany: EpisodeCategory (junction)
EpisodeCategory	EpisodeId, CategoryId	Junction (M:M)	Composite PK (EpisodeId + CategoryId)	BelongsTo: Episode, Category
Order	Id, UserId, EpisodeId, CreatedAt	User Library	Unique constraint on (UserId, EpisodeId). Episode must be published.	BelongsTo: User, Episode

4. API Contract

4.1 Authentication Endpoints

Method	Endpoint	Description	Auth	Response
POST	/api/auth/register	Register new user. Returns JWT access token.	None	200: { success, message, accessToken }
POST	/api/auth/login	Login with username + password. Sets refresh token cookie.	None	200: { success, message, accessToken }
POST	/api/auth/refresh	Refresh access token using HttpOnly cookie.	Cookie	200: { accessToken }

4.2 Episode Endpoints

Method	Endpoint	Description	Auth	Response
GET	/api/episodes/search?q=&page=&pageSize=	Search episodes by title, description, category	None	200: PagedResult<EpisodeSearchItemDto>
POST	/api/episodes/{id}/play	Increment play count for episode	None	200: { id, playCount } 404
POST	/api/publisher/episodes	Publisher creates new episode	Publisher/Admin	201: EpisodeDetailsDto
PUT	/api/publisher/episodes/{id}	Publisher updates their own episode	Publisher/Admin	200: EpisodeDetailsDto 403 404
DELETE	/api/publisher/episodes/{id}	Publisher deletes their own episode	Publisher/Admin	204 403 404
GET	/api/admin/episodes	Admin: list all episodes (including unpublished)	Admin	200: List<EpisodeDetailsDto>

4.3 Order Endpoints

Method	Endpoint	Description	Auth	Response
POST	/api/orders	Save/order an episode to user library	User	200: { id, episodeId, createdAt } 409 Conflict
GET	/api/orders	Get all orders for authenticated user	User	200: List<{ orderId, episodeId, createdAt }>
GET	/api/orders/my-episodes	Get full episode details for ordered episodes	User	200: List<EpisodeDto>
GET	/api/orders/episodes/{id}	Get specific ordered episode (auth check)	User	200: EpisodeDto 404

4.4 Admin & Category Endpoints

Method	Endpoint	Description	Auth	Response
GET	/api/admin/users	List all users	Admin	200: List<UserResponseDto>
POST	/api/admin/users	Create user with any role	Admin	201: UserResponseDto
PUT	/api/admin/users/{id}	Update user (including role)	Admin	200: UserResponseDto 404
DELETE	/api/admin/users/{id}	Delete user	Admin	204 404
GET	/api/categories	List all categories	None	200: List<CategoryDto>
POST	/api/categories	Create new category	Any	200: CategoryDto 409 Conflict
DELETE	/api/categories/{id}	Delete category (blocked if in use)	Any	204 400 (in use) 404

6. Non-Functional Requirements

Category	Requirement	Implementation
Performance	API responses < 500ms for paginated episode lists	AsNoTracking() on read-only queries. EF projections with .Select() to avoid over-fetching. Pagination capped at 50 items.
Security	Passwords never stored in plaintext. JWT expiry enforced.	BCrypt with salt for all password hashing. Access token: 15 min expiry. Refresh token: 7 days (HttpOnly cookie). Role claims in JWT.
Security	Publisher can only manage own episodes	PublisherId checked on all PUT/DELETE operations in PublisherController. Returns 403 if mismatch.
Availability	Platform accessible via public URL after deployment	FE and BE deployed as separate Azure Kubernetes pods. Horizontal scaling possible via AKS.
Maintainability	Code follows naming conventions and is decomposed	Controllers separated by concern (8 controllers). DTOs used for all input/output. Services extracted (CloudinaryService, UserCreateService).
Responsiveness	Frontend works on mobile and desktop	CSS media queries implemented. CSS framework Bootstrap used for responsive layout.